RUM Community Group

2025-09-12 Edition

Agenda

- Logistics
- Next Meeting
- Agenda
  - Request for feedback on w3c/navigation-timing/#193
  - Interop 2025 Updates
  - Interop 2026 Proposals
  - Unresponsive crash reports, Self Profiling

Logistics

- Meeting cadence
  - 2nd Friday of the Month
  - 60 minutes
  - 10am ET
- Agenda Document
  - (bit.ly/rumcg-agenda)
- Google Meet for Meetings
  - sub to rumcg-participants for invites
- Chat on Web Performance Slack
  `#w3c-rum-community-group`
  - (invites)
- public-rumcg@w3.org
  - (for discussions)

- RUM CG Github Repo
  - github.com/w3c-cg/rum
  - Group details
  - Links
  - Meeting minutes?
- RUM CG Tracking Project
  - github.com/orgs/w3c-cg/projects/1
- Meeting Minutes via fireflies.ai
  - AI summary + transcript will be posted after
  - Full video recording also available

Next Meeting

- Fri **October 10th** @ 10am EST
  - 2nd Friday of each month

- Topics?
  - *RUM Archive?*
  - *(propose your topic here!)*

- Submit & Vote topics @ bit.ly/rumcg-agenda

Topics Backlog

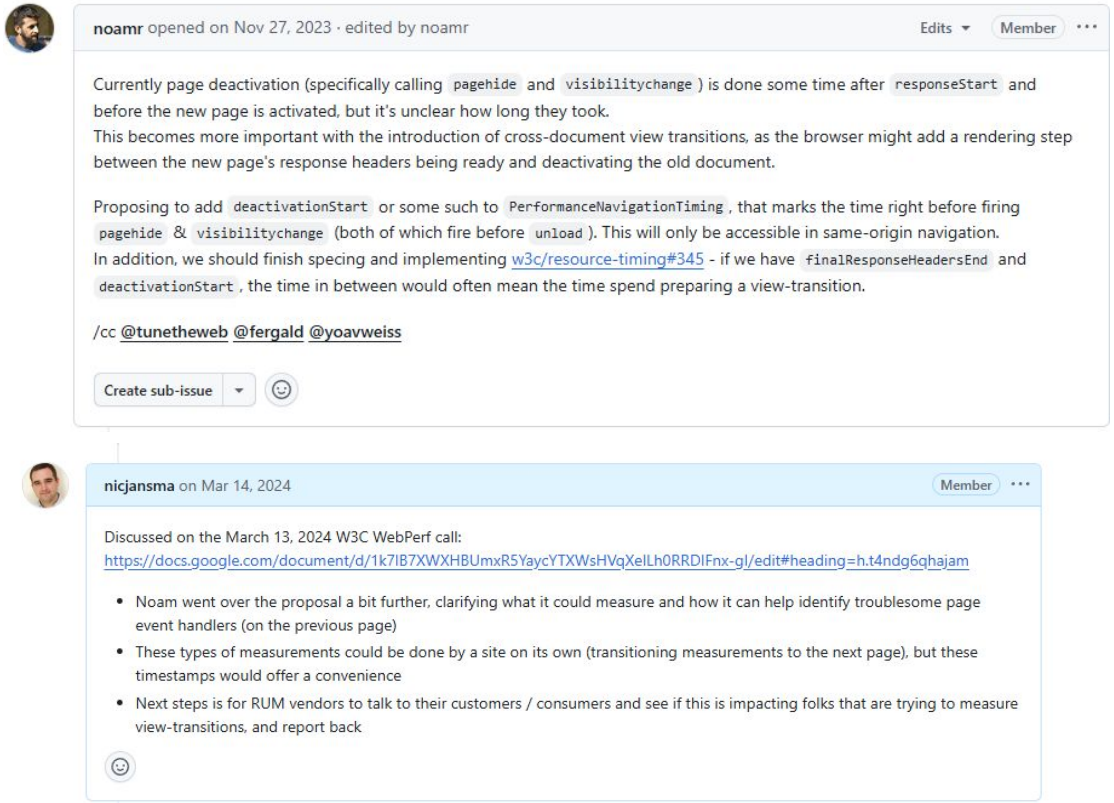
- ~~Better tracking/visibility into vendor positions (March 14)~~
- ~~Top tracking issues (e.g. this lighthouse issue) (March 14)~~
- ~~Baseline Project~~
- ~~Funding browser work for features~~
- ~~Header adoption (TAO, Server-timing)~~
- ~~Browser vendor presentations~~
- Outreach to other groups (i.e. 3rd Parties, CDNs)
- Ad Blockers & tracking protection
- Implementation best practices
  - When to fire the beacon/how to deliver
  - SPA support
- Making sure we don't step on each other's toes (i.e. clearing RT buffers, multiple vendors on a page, etc.)
- Open source
- Securing the beacon
- RUM Archive
- ~~Callstacks from input delays (Issac Gerges thread in #general)~~
- Container Timing awareness
- ServerTiming / trailers

Proposal: add timestamp for when "activation" or "dismissal" starts

[w3c/navigation-timing/#193](#)

Request: Is anyone here interested in measuring this?

Do we have any websites or customers that are seeking visibility into e.g. troublesome event handlers on previous page causing delays?



noamr opened on Nov 27, 2023 · edited by noamr — Edits — Member

Currently page deactivation (specifically calling `pagehide` and `visibilitychange`) is done some time after `responseStart` and before the new page is activated, but it's unclear how long they took.
This becomes more important with the introduction of cross-document view transitions, as the browser might add a rendering step between the new page's response headers being ready and deactivating the old document.

Proposing to add `deactivationStart` or some such to `PerformanceNavigationTiming`, that marks the time right before firing `pagehide` & `visibilitychange` (both of which fire before `unload`). This will only be accessible in same-origin navigation.
In addition, we should finish specing and implementing w3c/resource-timing#345 - if we have `finalResponseHeadersEnd` and `deactivationStart`, the time in between would often mean the time spend preparing a view-transition.

/cc @tunetheweb @fergald @yoavweiss

Create sub-issue

nicjansma on Mar 14, 2024 — Member

Discussed on the March 13, 2024 W3C WebPerf call:
https://docs.google.com/document/d/1k7IB7XWXHBUmxR5YaycYTXWsHVqXeILh0RRDIFnx-gI/edit#heading=h.t4ndg6qhajam

- Noam went over the proposal a bit further, clarifying what it could measure and how it can help identify troublesome page event handlers (on the previous page)
- These types of measurements could be done by a site on its own (transitioning measurements to the next page), but these timestamps would offer a convenience
- Next steps is for RUM vendors to talk to their customers / consumers and see if this is impacting folks that are trying to measure view-transitions, and report back

Interop 2025 Updates

- Announcements
  - https://webkit.org/blog/16458/announcing-interop-2025/
  - https://web.dev/blog/interop-2025
  - https://www.igalia.com/chats/interop2025
  - https://hacks.mozilla.org/2025/02/interop-2025/

- What is it?
  - *Collaboration between browser vendors and other platform implementors to provide users and web developers with high quality implementations of the web platform.*
  - *Each year we select a set of focus areas representing key areas where we want to improve interoperability.*

Interop 2025 Updates

Core Web Vitals

Web Vitals can help you quantify the experience of your site and identify opportunities to improve. The Web Vitals initiative aims to simplify the landscape, and help sites focus on the metrics that matter most, the Core Web Vitals.

Interop 2025 includes Largest Contentful Paint (LCP) and Interaction to Next Paint (INP) metrics by implementing the LargestContentfulPaint API and the Event Timing API across browsers. The Cumulative Layout Shift (CLS) metric is not in scope.

LCP API

Browser Support 

Source

Event Timing API (for INP)

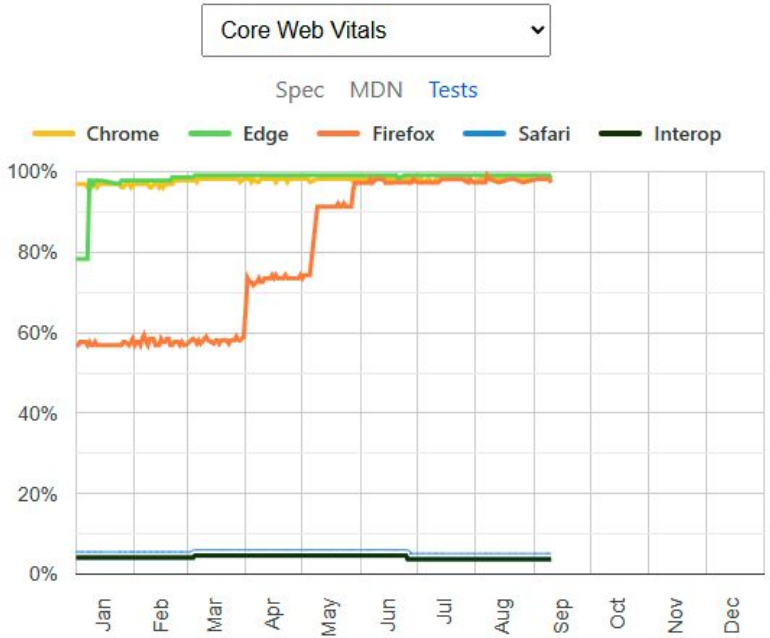
Browser Support 

Source

Interop 2025 Updates

https://wpt.fyi/interop-2025?feature=interop-2025-core-web-vitals

Proposal to revert navigation API test regression introduced by WebKit



Interop 2026 Proposals

https://github.com/web-platform-tests/interop/blob/main/proposal_guide.md

https://github.com/web-platform-tests/interop/blob/main/2026/README.md

Interop 2026 will open for proposals on September 4, 2025 and the overall timeline is as below. Please note that the timelines are subject to change.

- *Proposal submission window (~3 weeks): Sept 4th, 2025 through Sept 24th, 2025*
- *Proposal selection: Before December 11th 2025*
- *Scope of the Interop 2026 project published: the first half of February, 2026.*

Interop 2026 Proposals

Interesting proposals already:

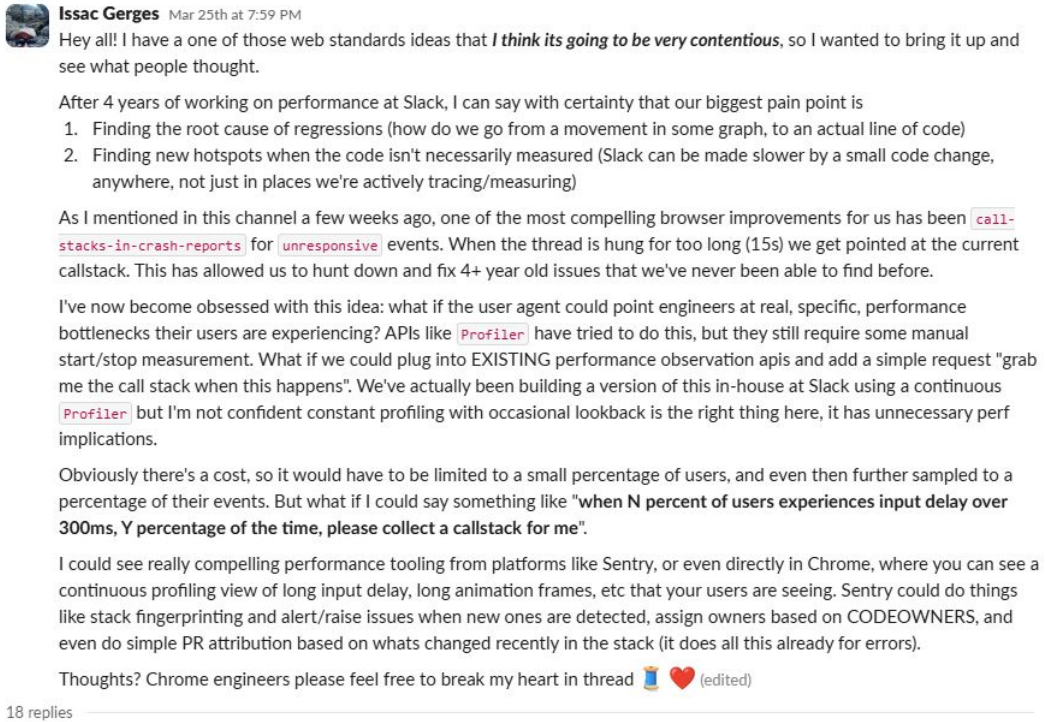
- Core Web Vitals: CLS / Layout Shift
- Long Animation Frames API
- requestIdleCallback
- JPEG XL image format
- fetchLater
- Speculation Rules?
- ResourceTiming (WPT)?
- CWV LCP + ET what's left?
- Signature-Based SRI?
- What else!?!?!?!

Next steps:

- "voting" by Dec 11th

Unresponsive crash reports + Self Profiling

- https://webperformance.slack.com/archives/C04BK7K1X/p1742947178736999



Issac Gerges Mar 25th at 7:59 PM

Hey all! I have a one of those web standards ideas that ***I think its going to be very contentious***, so I wanted to bring it up and see what people thought.

After 4 years of working on performance at Slack, I can say with certainty that our biggest pain point is

1. Finding the root cause of regressions (how do we go from a movement in some graph, to an actual line of code)
2. Finding new hotspots when the code isn't necessarily measured (Slack can be made slower by a small code change, anywhere, not just in places we're actively tracing/measuring)

As I mentioned in this channel a few weeks ago, one of the most compelling browser improvements for us has been `call-stacks-in-crash-reports` for `unresponsive` events. When the thread is hung for too long (15s) we get pointed at the current callstack. This has allowed us to hunt down and fix 4+ year old issues that we've never been able to find before.

I've now become obsessed with this idea: what if the user agent could point engineers at real, specific, performance bottlenecks their users are experiencing? APIs like `Profiler` have tried to do this, but they still require some manual start/stop measurement. What if we could plug into EXISTING performance observation apis and add a simple request "grab me the call stack when this happens". We've actually been building a version of this in-house at Slack using a continuous `Profiler` but I'm not confident constant profiling with occasional lookback is the right thing here, it has unnecessary perf implications.

Obviously there's a cost, so it would have to be limited to a small percentage of users, and even then further sampled to a percentage of their events. But what if I could say something like "**when N percent of users experiences input delay over 300ms, Y percentage of the time, please collect a callstack for me**".

I could see really compelling performance tooling from platforms like Sentry, or even directly in Chrome, where you can see a continuous profiling view of long input delay, long animation frames, etc that your users are seeing. Sentry could do things like stack fingerprinting and alert/raise issues when new ones are detected, assign owners based on CODEOWNERS, and even do simple PR attribution based on whats changed recently in the stack (it does all this already for errors).

Thoughts? Chrome engineers please feel free to break my heart in thread 🧵 ❤️ (edited)

18 replies